



NORTHERN UNIVERSITY

OF BUSINESS & TECHNOLOGY KHULNA

PROJECT REPORT

Smart Quiz Generator

An Intelligent, Role-Based Quiz Management System with AI-Assisted Question Generation

Course	CSE4204 — Mobile Computing Lab
Team / Section	Team 04 (T04), Section 8D — CSE4204-8D-T04
Department	Computer Science & Engineering (CSE)
Institution	Northern University of Business and Technology, Khulna
Submitted To	MD Riaz Mahmud, Assistant Professor, Dept. of CSE
Repository	github.com/theuglyhaxor/CSE4204-8D-T04-SMART-QUIZ-GENERATOR
Date of Submission	14 August 2026

SUBMITTED BY — TEAM CSE4204-8D-T04

MD Rohan	11220320958	Backend Developer, Full Stack
Sharmin Nahar Tumpa	11220320962	AI Integration
Pial Tarofdar	11220320965	Frontend Developer
Sanjana Athoy	11220320953	Technical Support

Table of Contents

Chapter 1 — Introduction	4
1.1 Background and Motivation	4
1.2 Problem Statement	4
1.3 Objectives	5
1.4 Scope of the Project	5
Chapter 2 — Requirement Analysis	6
2.1 User Roles and Stakeholders	6
2.2 Functional Requirements	6
2.3 Non-Functional Requirements	7
2.4 Use Cases	8
Chapter 3 — System Design	10
3.1 Technology Stack	10
3.2 System Architecture	10
3.3 Authentication and Authorization Design	11
Chapter 4 — Database Design	13
4.1 Entity-Relationship Diagram	13
4.2 Table Definitions	13
4.3 Deployed Schema Evidence	15
Chapter 5 — AI Integration	16

5.1 Multi-Provider Design	16
5.2 Document Ingestion Pipeline	16
5.3 End-to-End Generation Flow	16
Chapter 6 — Implementation	18
6.1 Backend Implementation	18
6.2 REST API Reference	18
6.3 PDF Export Engine	19
6.4 Frontend Implementation	20
Chapter 7 — Testing and Quality Assurance	22
7.1 Testing Strategy	22
7.2 Automated API Test Suite	22
7.3 Manual UI Verification and Bug Fixes	23
Chapter 8 — Results and System Screenshots	25
Chapter 9 — Challenges and Solutions	35
Chapter 10 — Conclusion and Future Work	36
References	37
Appendix A — Team Contributions	38
Appendix B — Running the Project Locally	39

CHAPTER 1

Introduction

1.1 Background and Motivation

Assessment is one of the most time-consuming responsibilities a teacher carries. Writing multiple-choice questions that are fair, unambiguous, and actually aligned with the material covered in class takes real effort, and it is work that has to be repeated for every topic, every term, and every cohort of students. At the same time, the tooling that most small institutions and individual educators have access to for running a quiz — spreadsheets, paper forms, generic form builders — was not designed for the job: it does not understand the difference between a teacher and a student, it cannot score anything automatically, and it certainly cannot write a question for you.

The **Smart Quiz Generator** was built as a response to that gap. It is a full-stack, role-aware quiz platform where a teacher can either write questions by hand or hand the system a source document — a lecture note, a PDF chapter, a plain-text summary — and have an AI model read that material and draft a complete set of multiple-choice questions, each with a marked correct answer and a short explanation. The teacher stays in the loop: every AI-generated quiz is created as an editable draft that must be reviewed and explicitly published before any student can see it. Students, in turn, get a clean, timed quiz-taking interface with instant, automatic scoring and a personal history of every attempt they have made.

The project was developed by Team CSE4204-8D-T04 as the practical component of the CSE4204 Mobile Computing Lab course, and doubles as a demonstration of how a modern web stack — a Django REST API on the backend and a React single-page application on the frontend — can be combined with a commercial large language model to automate a genuinely tedious part of an educator's workload.

1.2 Problem Statement

Traditional quiz creation and administration is manual and slow: educators write every question by hand, track submissions in spreadsheets or on paper, and grade multiple-choice answers themselves even though the process is entirely mechanical. This has three concrete consequences that the project set out to address:

- **High authoring cost.** Generating a bank of good multiple-choice questions from a chapter of course material can take longer than teaching the chapter itself, discouraging frequent, low-stakes assessment.
- **No secure, role-aware workflow.** Generic tools do not distinguish between the person setting the assessment and the person taking it, so answer keys, draft quizzes, and other students' scores are difficult to keep confidential without extra manual process.
- **Delayed feedback.** Manually marking submissions means students wait — sometimes days — to find out what they got wrong and why, by which point the learning moment has often passed.

1.3 Objectives

The system was designed to meet the following goals:

- Enable quiz creation both manually and automatically, by generating questions with an AI model from an uploaded document or a typed topic/syllabus description.
- Enforce strict, role-based access control so that only teachers can author, publish, or delete quizzes and view submissions, while students can only take active quizzes and see their own results.
- Score every submission automatically and immediately, with a per-question explanation of the correct answer so students receive useful feedback the moment they submit.
- Support two interchangeable AI providers (Google Gemini and Anthropic Claude) rather than depending on a single vendor for question generation.
- Produce print-ready PDF exports of any quiz — either a clean student handout or a teacher's answer-key copy — directly from the data already stored in the database.
- Expose the whole feature set through a documented, versioned REST API so the frontend (and, in principle, any other client) consumes the same contract.

1.4 Scope of the Project

The delivered system is a complete, working full-stack application, not a prototype of one component. It covers the entire lifecycle of a quiz: creation (manual or AI-assisted), review and publishing, document upload and parsing, student discovery and attempt submission, automatic scoring, attempt history, a reusable question bank across quizzes, and PDF export. Authentication, authorization, and the underlying relational schema are all implemented and enforced server-side, not merely assumed by the client.

What is explicitly out of scope for this iteration: production infrastructure concerns such as a reverse proxy/load balancer, a caching layer, and containerised deployment; a native mobile application (the REST API is client-agnostic and could support one); real-time features such as live leaderboards or WebSocket push notifications; and a dedicated system-administrator role beyond the teacher/student split. These are discussed as future work in Chapter 10.

CHAPTER 2

Requirement Analysis

2.1 User Roles and Stakeholders

The system recognises two authenticated roles, implemented as Django auth groups (`teacher` and `student`) that every protected endpoint checks before granting access.

Teacher

- Creates, updates, publishes, and deletes their own quizzes.
- Adds questions manually, or generates them from a topic or an uploaded document via AI.
- Maintains a cross-quiz question bank and can review every student attempt on their quizzes.
- Exports any quiz as a PDF, with or without the answer key.

Student

- Browses only *active* (published) quizzes.
- Retrieves questions with the correct answers stripped out server-side.
- Submits answers once, receives an immediate score and a per-question explanation, and can review their own attempt history at any time.

Role is never trusted from client input. The permission layer (`quiz_api/permissions.py`) derives the caller's role from their Django group membership on every request, so a student cannot elevate themselves by editing a request payload.

2.2 Functional Requirements

ID	Requirement
FR-01	Users register with a username, optional email, password, and a declared role (teacher or student).
FR-02	The system authenticates users with JSON Web Tokens (access + refresh) and blacklists refresh tokens on logout.
FR-03	Teachers can create, list, retrieve, update, and delete quizzes they own.
FR-04	New quizzes default to a draft state (<code>is_active=false</code>) and are invisible to students until explicitly published.
FR-05	Teachers can add, edit, delete, and reorder multiple-choice questions (four options, one correct answer, optional explanation) within a quiz.

FR-06	Teachers can upload a source document (PDF, TXT, MD, CSV, or JSON) and have its text extracted server-side.
FR-07	Teachers can trigger AI question generation from a topic, a syllabus/document text, a difficulty level, a target question count, and optional free-text instructions, choosing between the Gemini and Claude providers.
FR-08	An AI generation call persists the resulting quiz and questions immediately, as a draft, so the teacher is reviewing already-saved data rather than an ephemeral preview.
FR-09	Students can list active quizzes and retrieve a quiz's questions with the <code>correct_option</code> and <code>explanation</code> fields removed from the response.
FR-10	Students can submit one set of answers per attempt; the server computes the score, persists the attempt, and returns per-question correctness and explanations in the same response.
FR-11	Students can list their own past attempts; they cannot list or read another student's attempts.
FR-12	Teachers can list every attempt submitted against a quiz they own.
FR-13	Any quiz can be exported as an A4 PDF, either as a full answer-key copy (teacher) or a clean handout with no answers shown (student or teacher).
FR-14	A lightweight <code>/meta/</code> and <code>/stats/</code> endpoint expose team identity and per-user aggregate counters (quizzes created, questions written, attempts, average score) used by the dashboard and profile pages.

2.3 Non-Functional Requirements

Category	Requirement
Security	Passwords are hashed with Django's PBKDF2-SHA256; JWT access tokens expire after 60 minutes and refresh tokens after 24 hours with rotation and blacklisting enabled; every teacher-only endpoint verifies group membership server-side.
Data isolation	A student can only ever read their own attempts; a teacher can only update or delete quizzes they created (enforced at the queryset level, not just the UI).
Portability	The database layer is swappable between SQLite (zero-setup local development) and MySQL/MariaDB (production) via a single <code>DB_ENGINE</code> environment variable, with no code changes.
Resilience of AI features	Two independent AI providers are supported so question generation is not a single point of failure; the renderer used for AI-generated content degrades gracefully to a transliterated ASCII font if no Unicode-capable TrueType font is available on the host.
Usability	The frontend is a responsive single-page application with light and dark themes, consistent loading/empty/error states across every data-driven page, and role-based navigation that never shows a control the signed-in user cannot use.

Maintainability	The PDF renderer used for quiz exports is a single shared module called from both the live API endpoint and an offline CLI, guaranteeing the two never drift apart.
Documentation	Every REST endpoint is exercised by an executable diagnostic script (<code>system_check.py</code>) that doubles as living documentation of expected request/response shapes.

2.4 Use Cases

The five use cases below describe the primary interaction flows exercised by the two roles.

UC-1 — Teacher generates a quiz with AI

Actor: Teacher · **Precondition:** teacher is authenticated.

1. Teacher opens *Create Quiz* and enters a topic, or attaches a source document.
2. If a document was attached, the frontend first calls `POST /api/documents/parse/` to extract its text.
3. The frontend calls `POST /api/ai/generate-quiz/` with the topic/syllabus text, difficulty, question count, duration, optional instructions, and provider choice.
4. The backend builds a structured prompt, calls the selected AI provider, validates the returned JSON against the expected question schema, and persists a new draft `Quiz` with its `Question` rows in a single transaction.
5. The teacher reviews the generated questions in the preview pane and either publishes the quiz, discards it, or exports it to PDF.

UC-2 — Teacher publishes and exports a quiz

Actor: Teacher · **Precondition:** a draft quiz with at least one question exists.

1. Teacher clicks *Publish to students*, which sends `PATCH /api/quizzes/<id>/` with `is_active=true`.
2. Teacher downloads a handout or answer-key PDF via `GET /api/quizzes/<id>/export-pdf/`.

UC-3 — Student takes a quiz

Actor: Student · **Precondition:** at least one quiz is active.

1. Student browses the list of active quizzes and opens one.
2. The frontend fetches student-safe questions (no answers) and starts a countdown timer based on the quiz's configured duration.
3. Student selects one option per question and submits — either voluntarily or automatically when time runs out.
4. The backend scores the submission, stores the attempt against the student's account, and returns the score together with the correct answer and explanation for every question.

UC-4 — Teacher reviews submissions

Actor: Teacher.

1. Teacher opens a quiz's attempt list (`GET /api/quizzes/<id>/attempts/`).
2. The system returns every student's score, response set, and submission timestamp for that quiz.

UC-5 — Student reviews their own history

Actor: Student.

1. Student opens *My Attempts*, which calls `GET /api/attempts/me/` .
2. The system returns only attempts owned by the authenticated student, each with its score and percentage, letting the student track improvement over time.

CHAPTER 3

System Design

3.1 Technology Stack

Layer	Technology
Backend framework	Django 4.2 + Django REST Framework, Python 3.9+
Authentication	JSON Web Tokens via <code>django-rest-framework-simplejwt</code> (access + refresh, rotation, blacklist)
Database	SQLite (default, zero-setup development) or MySQL 5.7+ / MariaDB 10.3+ (production) via PyMySQL, selected with the <code>DB_ENGINE</code> environment variable
Cross-origin support	<code>django-cors-headers</code> , configured for the Vite dev origin
Document parsing	<code>pypdf</code> for PDF text extraction; plain UTF-8 decoding for TXT/MD/CSV/JSON
AI providers	Google Gemini (REST) and Anthropic Claude (official SDK), selectable per request
PDF generation	<code>fpdf2</code> , with a Unicode TrueType font when available and safe ASCII transliteration as a fallback
Frontend framework	React 18 (function components + hooks), React Router 7, built with Vite 5
Frontend icons	<code>lucide-react</code>
API testing	A Postman collection plus an executable Python diagnostic suite (<code>system_check.py</code> , <code>manual_api_test.py</code>)

3.2 System Architecture

The application follows a conventional three-tier structure: a React single-page application, a stateless Django REST API, and a relational database, with two external AI services consulted only for the question generation feature. In local development the Vite dev server proxies every `/api/*` request to Django on port 8000, so the browser only ever talks to one origin and CORS is not a concern outside of production. In production, the same static frontend build can be served by any static host or reverse proxy in front of the Django API.

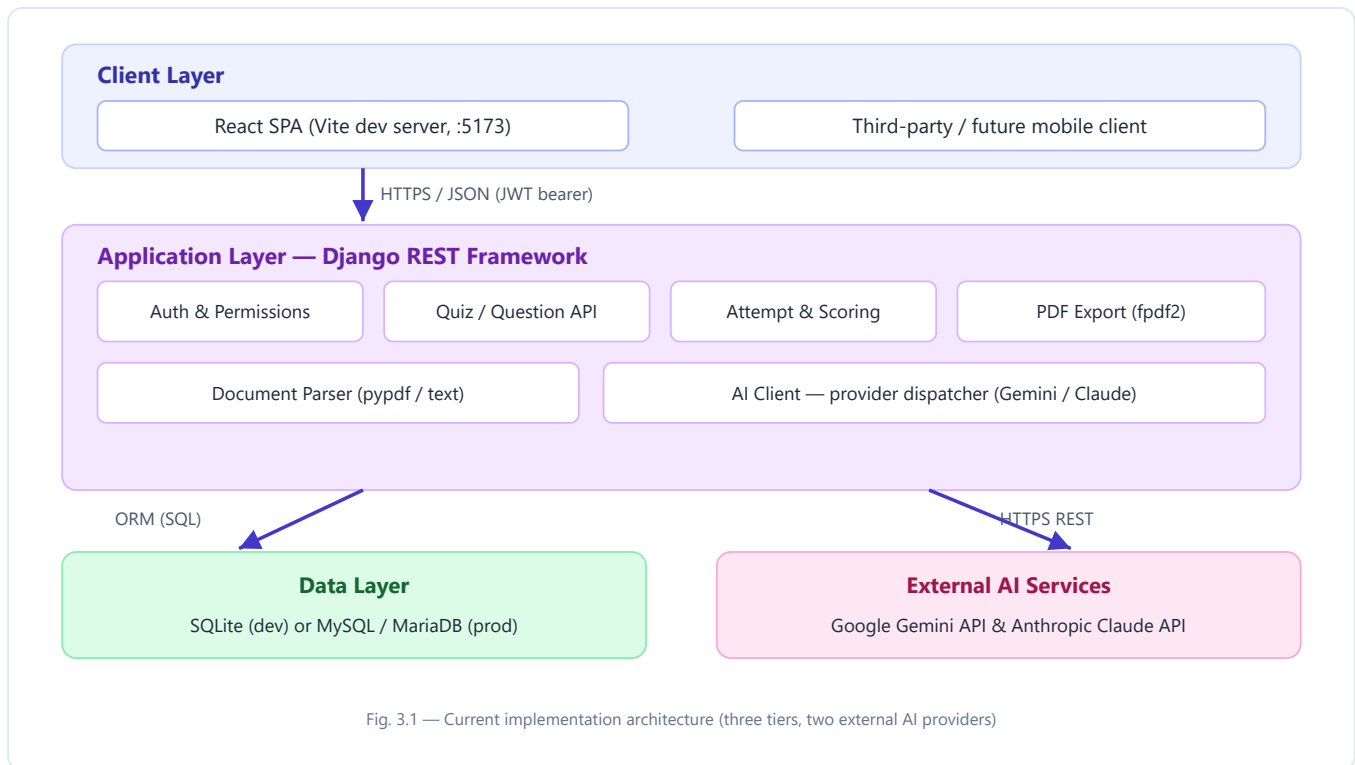


Figure 3.1. System architecture as implemented: a React SPA talks to a stateless Django REST API over JSON; the API owns all persistence and only calls out to Gemini or Claude for the AI generation feature.

Application layer responsibilities

- **Auth & Permissions** — JWT issuance/refresh/logout and the `get_user_role()` helper that every teacher-only or student-only view depends on.
- **Quiz / Question API** — CRUD for quizzes and their nested questions, plus the student-safe question view that strips answer fields.
- **Attempt & Scoring** — validates a submission against the stored questions, computes the score server-side (never trusting a client-submitted score), and persists the attempt.
- **Document Parser** — converts an uploaded PDF/TXT/MD/CSV/JSON file into clean text used to seed AI generation.
- **AI Client** — a single `generate_quiz()` dispatcher that picks Gemini or Claude based on the request (or the `AI_PROVIDER` environment default) and normalises both providers' output into the same question schema.
- **PDF Export** — a shared renderer used identically by the live download endpoint and an offline quality-review CLI, so the two can never visually diverge.

3.3 Authentication and Authorization Design

Authentication is entirely stateless: on login, the API issues a short-lived access token (60 minutes) and a longer-lived refresh token (24 hours). The frontend attaches the access token as a `Bearer` header on every request and transparently calls `POST /api/auth/token/refresh/` when it expires. Refresh tokens rotate on every use and the previous token is blacklisted, so a stolen refresh token cannot be replayed after it has legitimately been used once. Logging out blacklists the current refresh token outright.

Authorization is role-based rather than object-list-based: a request is authenticated first (is this a valid token?), then authorized by role (is this user in the `teacher` or `student` Django group?), and finally, for object-level operations such as editing a quiz, checked against ownership (`quiz.created_by == request.user`). All three checks happen server-side on every request; the frontend's role-based navigation and route guards are a usability layer only; they are not the security boundary.

CHAPTER 4

Database Design

4.1 Entity-Relationship Diagram

The schema is intentionally small: three application tables sit on top of Django's built-in authentication tables and the JWT refresh-token blacklist tables that `django-rest-framework-simplejwt` manages automatically.

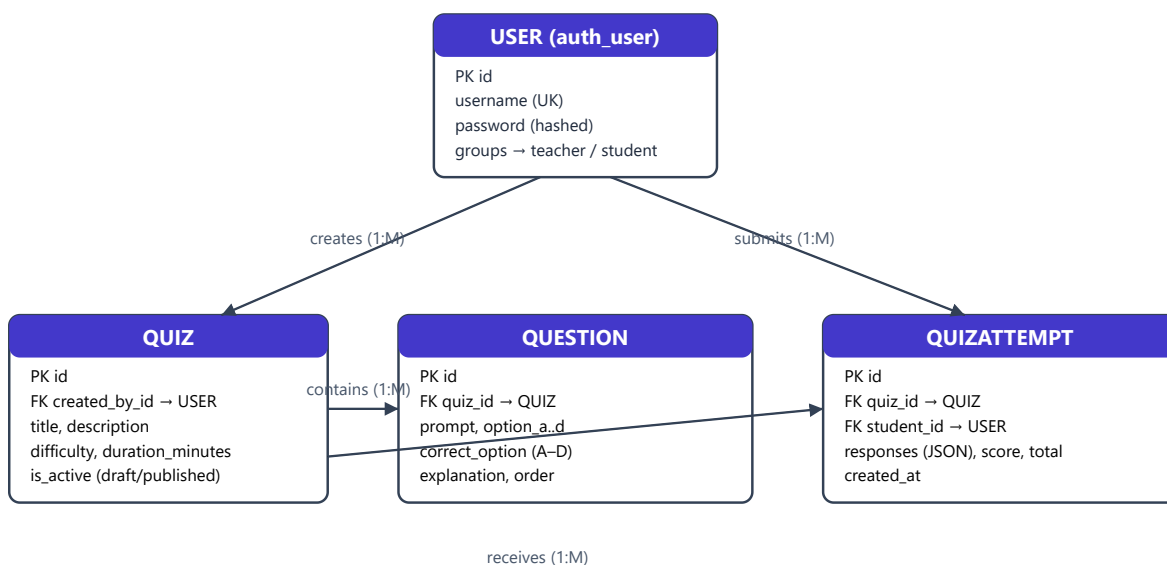


Fig. 4.1 — Entity-relationship diagram of the quiz_api schema

Figure 4.1. QUIZ 1:M QUESTION (cascade delete), QUIZ 1:M QUIZATTEMPT (cascade delete), USER 1:M QUIZ (owner, nullable), USER 1:M QUIZATTEMPT (submitter).

4.2 Table Definitions

quiz_api_quiz — quiz metadata and configuration

Column	Type	Notes
id	INTEGER	Primary key, auto-increment.
title	VARCHAR(255)	Required.
description	LONGTEXT	Optional instructions shown above the questions.
difficulty	VARCHAR(50)	Easy / Medium / Hard, default Medium.
duration_minutes	INT UNSIGNED	Default 5; drives the countdown timer on the student side.

is_active	TINYINT(1)	Default 0 (draft). Only true = visible to students.
created_by_id	INT, FK → auth_user	Nullable; <code>ON DELETE SET NULL</code> so a deleted user does not cascade-delete their quizzes.
created_at / updated_at	DATETIME(6)	Auto-managed timestamps.

quiz_api_question — one multiple-choice question, belongs to a quiz

Column	Type	Notes
id	INTEGER	Primary key.
quiz_id	INT, FK → quiz	<code>ON DELETE CASCADE</code> .
prompt	LONGTEXT	The question text.
option_a..option_d	VARCHAR(255) × 4	Exactly four fixed options — no variable-length option sets.
correct_option	VARCHAR(1)	Constrained to <code>A – D</code> at both the application and database (CHECK) level.
explanation	LONGTEXT	Optional; shown to students only after they submit.
order	INT UNSIGNED	Display order within the quiz.

quiz_api_quizattempt — one student's submission for a quiz

Column	Type	Notes
id	INTEGER	Primary key.
quiz_id	INT, FK → quiz	<code>ON DELETE CASCADE</code> .
student_id	INT, FK → auth_user	Nullable; the field authorization is actually checked against.
student_name	VARCHAR(255)	Denormalised display label derived server-side from the authenticated user, never taken from the request body.
responses	JSON	Per-question selected option and correctness, embedded rather than normalised into a separate answers table.
score / total	INT UNSIGNED	Computed and stored server-side at submission time.
created_at	DATETIME(6)	Attempt timestamp, used for history ordering.

Authentication reuses Django's own `auth_user`, `auth_group`, and `auth_user_groups` tables rather than a bespoke user model, and the JWT layer adds `token_blacklist_outstandingtoken` and

`token_blacklist_blacklisttoken` to track and revoke refresh tokens. Reusing Django's battle-tested auth tables — instead of hand-rolling a `users` table and password column — was a deliberate choice to avoid re-implementing password hashing and group management from scratch.

4.3 Deployed Schema Evidence

The reference DDL in `database/schema.sql` mirrors exactly what Django's migrations produce; it exists for review purposes only; the source of truth is always `python manage.py migrate`. Figure 4.2 shows the schema materialised on a live MySQL instance during development, confirming that the tables, engine (InnoDB), and character set (utf8mb4) match the design above.

Table	Action	Rows	Type	Collation	Size	Overhead
☐ authtoken_token	★ Browse Structure Search Insert Empty Drop	10	InnoDB	utf8mb4_unicode_ci	32.0 KiB	-
☐ auth_group	★ Browse Structure Search Insert Empty Drop	2	InnoDB	utf8mb4_unicode_ci	32.0 KiB	-
☐ auth_group_permissions	★ Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	48.0 KiB	-
☐ auth_permission	★ Browse Structure Search Insert Empty Drop	44	InnoDB	utf8mb4_unicode_ci	32.0 KiB	-
☐ auth_user	★ Browse Structure Search Insert Empty Drop	11	InnoDB	utf8mb4_unicode_ci	32.0 KiB	-
☐ auth_user_groups	★ Browse Structure Search Insert Empty Drop	10	InnoDB	utf8mb4_unicode_ci	48.0 KiB	-
☐ auth_user_user_permissions	★ Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	48.0 KiB	-
☐ django_admin_log	★ Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	48.0 KiB	-
☐ django_content_type	★ Browse Structure Search Insert Empty Drop	11	InnoDB	utf8mb4_unicode_ci	48.0 KiB	-
☐ django_migrations	★ Browse Structure Search Insert Empty Drop	24	InnoDB	utf8mb4_unicode_ci	16.0 KiB	-
☐ django_session	★ Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	32.0 KiB	-
☐ quiz_api_question	★ Browse Structure Search Insert Empty Drop	5	InnoDB	utf8mb4_unicode_ci	32.0 KiB	-
☐ quiz_api_quiz	★ Browse Structure Search Insert Empty Drop	3	InnoDB	utf8mb4_unicode_ci	16.0 KiB	-
☐ quiz_api_quizattempt	★ Browse Structure Search Insert Empty Drop	3	InnoDB	utf8mb4_unicode_ci	32.0 KiB	-
14 tables	Sum	123	InnoDB	utf8mb4_unicode_ci	496.0 KiB	0 B

Figure 4.2. The deployed schema on MySQL, viewed through phpMyAdmin — 14 tables in total: the three `quiz_api_*` application tables plus Django's own auth and JWT-blacklist tables.

CHAPTER 5

AI Integration

5.1 Multi-Provider Design

Question generation is deliberately not tied to a single AI vendor. A thin dispatcher, `ai_integration/providers.py`, exposes one function — `generate_quiz(payload, provider=None)` — that the rest of the backend calls without knowing or caring which model actually answers. The provider is chosen, in order of precedence, from an explicit argument on the request, the `AI_PROVIDER` environment variable, or a default of Gemini:

```
provider = (provider or os.environ.get("AI_PROVIDER") or "gemini").strip().lower()
```

Passing `"gemini"` routes to `generate_quiz_with_gemini()`; `"claude"` or `"anthropic"` routes to `generate_quiz_with_claude()`. Both implementations return the same normalised question shape, so nothing downstream — persistence, the PDF renderer, the frontend preview — needs to know which provider produced a given quiz.

This matters for reliability as much as for choice: if one vendor has an outage, a rate-limit incident, or a pricing change, the application is not blocked — a teacher can simply pick the other provider from a dropdown on the Create Quiz page and generate the same quiz a second time.

5.2 Document Ingestion Pipeline

When a teacher attaches a source document instead of (or alongside) typing a topic, the frontend first sends it to `POST /api/documents/parse/`, which is handled entirely independently of the AI call:

1. The file extension decides the extraction strategy: PDF files go through `pypdf.PdfReader`, concatenating the extracted text of every page; TXT, MD, CSV, and JSON files are decoded directly as UTF-8.
2. The extracted text is normalised (trailing whitespace stripped per line) and rejected with a clear error if it turns out to contain no readable text at all — for example, a scanned, image-only PDF with no embedded text layer.
3. The endpoint returns the cleaned text together with a word count, which the frontend shows to the teacher ("Read 1,842 words from lecture-04.pdf...") before the AI call even starts.

Only after this step succeeds does the frontend call the generation endpoint, passing the extracted text as the `syllabus` field. Separating parsing from generation keeps the AI prompt-construction code free of any file-format concerns and makes the parse step independently testable and reusable — the same parser also powers a teacher-facing "read this document" preview outside the quiz-generation flow.

5.3 End-to-End Generation Flow

1. Teacher fills in the Create Quiz form: topic and/or document, difficulty, question count, target duration, optional free-text instructions, and an AI provider (or leaves it on the server default).
2. The frontend calls `POST /api/ai/generate-quiz/`. The view validates the numeric inputs (question count 1–50, a positive duration) before spending an AI call on a request that would fail anyway.
3. The backend constructs a provider-appropriate prompt from the topic/syllabus text, difficulty, and any extra instructions, and calls `generate_quiz()`.
4. The provider's raw response is parsed and validated against the expected schema — each question needs a prompt, four options, a correct option in {A,B,C,D}, and (ideally) an explanation — before anything is written to the database.
5. On success, a new `Quiz` (draft, `is_active=false`) and its `Question` rows are created in one step and returned to the frontend, which immediately shows them in the review pane.
6. The teacher can now edit, discard, publish, or export the AI-authored quiz exactly like a manually created one — the AI is a first draft, not a black box the teacher has to trust blindly.

Because generation persists the quiz as a draft rather than returning an ephemeral preview, an AI call that a teacher decides not to keep is explicitly discarded (deleting the draft), rather than simply forgotten — the same review discipline applies to AI-authored and hand-written content alike.

CHAPTER 6

Implementation

6.1 Backend Implementation

The backend is a single Django project (`smart_quiz_backend`) with one application, `quiz_api`, organised conventionally:

`models.py` — Quiz, Question, QuizAttempt

`serializers.py` — DRF serializers, incl. a student-safe question serializer

`views.py` — API views/viewsets for every endpoint

`permissions.py` — role derivation and the three permission classes

`urls.py` — URL routing, all under `/api/`

`pdf.py` — the shared quiz → PDF renderer

`admin.py` — Django admin registration for quick data inspection

`ai_integration/` — provider dispatcher, Gemini + Claude clients, prompts, document parsing, response validation

Role checks are centralised in a single helper rather than duplicated per view:

`get_user_role(user)` returns `"teacher"`, `"student"`, or `None` based on Django group membership, and three tiny permission classes (`IsTeacherUser`, `IsStudentUser`, `IsTeacherOrStudentUser`) wrap it for DRF's `permission_classes`. Every view declares its required role explicitly instead of relying on a shared default.

Scoring is a good example of why the score is never trusted from the client: `QuizSubmitView` loads the quiz's real questions from the database, walks the submitted answers, compares each one against the stored `correct_option`, and only then computes and persists the score — a student cannot influence their own score by tampering with the request body beyond choosing which options to select.

6.2 REST API Reference

Method	Endpoint	Purpose
GET	<code>/api/meta/</code>	Team/course identity used by the frontend footer.
GET	<code>/api/stats/</code>	Per-user aggregate counters for the dashboard/profile.
POST	<code>/api/auth/register/</code>	Create an account with a declared role.
POST	<code>/api/auth/login/</code>	Exchange credentials for a JWT access/refresh pair.

POST	<code>/api/auth/logout/</code>	Blacklist the current refresh token.
GET	<code>/api/auth/me/</code>	The authenticated user's own profile.
POST	<code>/api/auth/token/refresh/</code>	Rotate an access token from a valid refresh token.
GET / POST	<code>/api/quizzes/</code>	List quizzes (role-scoped) / create a quiz (teacher).
GET / PUT / PATCH / DELETE	<code>/api/quizzes/<id>/</code>	Retrieve, update, or delete a single quiz.
GET / POST	<code>/api/quizzes/<id>/questions/</code>	List or add questions on a quiz (teacher).
GET	<code>/api/quizzes/<id>/student-questions/</code>	Answer-stripped questions for a student taking the quiz.
GET	<code>/api/quizzes/<id>/export-pdf/</code>	Download the quiz as an A4 PDF (handout or answer key).
POST	<code>/api/quizzes/<id>/submit/</code>	Submit answers; returns the computed score and per-question feedback.
GET	<code>/api/quizzes/<id>/attempts/</code>	List every attempt on a quiz (teacher, owner-only).
GET / POST	<code>/api/questions/</code>	Cross-quiz question bank listing / creation.
GET / PUT / PATCH / DELETE	<code>/api/questions/<id>/</code>	Retrieve, update, or delete a single question.
GET	<code>/api/attempts/me/</code>	The authenticated student's own attempt history.
POST	<code>/api/documents/parse/</code>	Extract text from an uploaded PDF/TXT/MD/CSV/JSON file.
POST	<code>/api/ai/generate-quiz/</code>	Generate and persist a draft quiz with the selected AI provider.

6.3 PDF Export Engine

`quiz_api/pdf.py` is a single renderer shared by both the live download endpoint and an offline CLI (`ai_integration/generate_sample_quizzes_pdf.py`) used to spot-check freshly generated quizzes without going through the API — guaranteeing the PDF a teacher downloads from the running app is

byte-for-byte the same layout the offline tool produces. Every document is rendered at A4 page size (`FPDF(orientation="P", unit="mm", format="A4")`) with a 16 mm margin and automatic page breaks reserved for a persistent footer.

- **Full-bleed title band** with the quiz title, rendered in the project's indigo brand colour, followed by a metadata strip (difficulty, question count, duration, generation date, and AI model when applicable).
- **Numbered question blocks** that measure their own required height before drawing (via a dry-run of `multi_cell`) so a question is never split mid-way across a page break, and highlight the correct option in green when the answer key is requested.
- **A dedicated answer-key page**, generated only for the teacher's copy, listing every question's correct letter and text.
- **A closing team-identity block** naming every team member, their student ID, and role — the same footer format reused across this very report's cover page.
- **Font fallback**: the renderer looks for a Unicode TrueType font on the host (Segoe UI or Arial on Windows, DejaVu Sans on Linux) so curly quotes and special characters from AI-generated text render correctly; if none is found, it transliterates the handful of common special characters and falls back to the built-in Helvetica font rather than crashing the export.

A query parameter distinguishes the two audiences: `?answers=false` renders a clean student handout with no correct-option highlighting, no explanations, and no answer key page; the default renders the full teacher copy. The endpoint additionally refuses to give a student the answer key under any circumstance, regardless of what the query string requests, and refuses to export an inactive quiz to a student at all.

6.4 Frontend Implementation

The frontend is a React single-page application built with Vite, routed with React Router 7. A single `AuthProvider` holds the signed-in user and exposes `isTeacher` / `isStudent` flags; a `ProtectedRoute` component wraps every role-specific route and redirects unauthenticated or wrong-role visitors before the page even mounts.

Route	Role	Page
<code>/login</code> , <code>/register</code>	Public	Authentication
<code>/</code>	Teacher (else redirects to <code>/student</code>)	Dashboard — quiz overview, stat cards, quiz table
<code>/create-quiz</code>	Teacher	Quiz configuration form + AI generation + live preview
<code>/question-bank</code>	Teacher	Cross-quiz question listing, filtering, and manual add
<code>/student</code>	Student	Active quiz list

<code>/quiz/:id</code>	Student	Timed quiz-taking interface and result/review screen
<code>/attempts</code>	Student	Personal attempt history
<code>/profile</code> , <code>/settings</code>	Any signed-in user	Account details and preferences

All data-driven pages share the same shell — a fixed sidebar, a top navigation bar with a dark-mode toggle, and a content area — and the same visual language for loading spinners, empty states, and error banners, defined once in a shared stylesheet rather than re-implemented per page. Every write action that costs the user something (discarding a generated quiz, deleting a question) is confirmed before it fires.

The Create Quiz flow

This is the feature that most directly demonstrates the AI integration end to end, and was also the subject of the UI regression documented in Section 7.3: a two-panel layout where the left panel (`QuizConfig`) collects the generation parameters and the right panel (`QuizPreview`) progresses through three states — an empty prompt, a generating spinner, and the reviewable question list with publish / export / discard actions once generation succeeds.

CHAPTER 7

Testing and Quality Assurance

7.1 Testing Strategy

Testing combined three complementary layers: an executable API diagnostic suite that exercises every endpoint end to end against a real running server, a Postman collection covering the same surface for manual exploration, and hands-on browser verification of the frontend using an automated browser (Playwright) driving the actual running application rather than relying on static code review alone.

7.2 Automated API Test Suite

`backend/system_check.py` runs a full, ordered scenario against a live server — connectivity, registration of both a teacher and a student, duplicate-username rejection, login with correct and incorrect passwords, permission checks (a student cannot create a quiz; an anonymous request cannot either), quiz and question CRUD, student-safe question retrieval (answers hidden), attempt submission and scoring, teacher visibility into submissions, document parsing, AI quiz generation, JWT refresh, and logout/blacklist behaviour — and reports a PASS/FAIL/SKIP verdict for every step. A companion script, `capture_screenshots.py`, re-runs the identical suite and renders a Postman-style request/response image for each step, which produced the 28-step evidence trail stored in `documentation/api_screenshots/` and summarised below.

#	Step
01	Protected route rejected without a token
02– 04	Teacher & student registration; duplicate username rejected
05– 06	Login with correct password succeeds, wrong password fails
07– 08	A student, and an anonymous request, cannot create a quiz
09	Teacher creates a quiz
10– 12	Teacher adds three questions with answers A/B/C
13– 15	List quizzes, retrieve one, patch its title
16	Student sees questions with answers hidden
17– 19	Student submits an attempt and scores 2/3; teacher can view attempts; the student cannot view others'

20– 21	Teacher parses an uploaded document; a student is refused the same endpoint
22	AI-generated quiz created end to end
23– 26	JWT refresh; logout without a token rejected; logout blacklists the refresh token; a blacklisted token is rejected on reuse
27– 28	Cleanup — quizzes created during the run are deleted

7.3 Manual UI Verification and Bug Fixes

Alongside the API suite, the running frontend was driven directly in a real browser to verify that every page renders correctly for both roles, in both light and dark mode. This process surfaced and fixed a concrete, previously-unreported layout defect on the Create Quiz page, which is documented here as a record of the debugging process rather than glossed over.

Defect: Create Quiz page rendered with no layout

The Create Quiz page's outer shell (the fixed sidebar plus a content column offset to sit beside it) is defined once, in `Dashboard.css`, and every other page that uses it — the Dashboard itself, Question Bank, My Attempts, the profile page, and the quiz-taking page — explicitly imports that stylesheet. `CreateQuiz.jsx` was missing that one import. Without it, the sidebar and content area fell back to plain unstyled block layout: no flex row, no sidebar-width offset, and no page background, so the two configuration/preview panels rendered stacked and misaligned instead of side by side.

Fixing the missing import corrected the outer shell, but exposed a second, subtler defect underneath it: the page's two panels (`.left-panel` / `.right-panel`) collapsed to roughly 45% and 55% of their intended width. Because the frontend's router imports every page component eagerly at start-up (there is no route-level code-splitting), every page's CSS is loaded into the document at once, regardless of which route is active. `Register.css` happened to define its own `.left-panel` / `.right-panel` rules — sized at 45%/55% for its own two-column sign-up layout — using the exact same, unscoped class names. Because Register's stylesheet loaded later in the module graph, its width rules silently overrode Create Quiz's, shrinking both panels to a fraction of the page. The same investigation found the identical pattern causing a decorative "AI-Powered Learning" badge on the Login page to be flung to the corner of the viewport (colliding with an unrelated `.badge` rule in the navbar's notification-dot styling) and the Login page's logo icon to render invisible (white-on-white, colliding with the sidebar's own `.logo-icon` rule).

The fix in every case was the same: rename the colliding classes to page-scoped names (`.createquiz-left`, `.login-badge`, `.login-logo-icon`, and so on) rather than patching the symptom, so the same class of bug cannot silently resurface if another page ever reuses a generic name like `.badge` or `.logo` again. Two genuinely dead stylesheet blocks discovered in the process — a pre-refactor `.card-header` rule in `Dashboard.css` that no longer matched anything in `Dashboard.jsx`, and an entire unused

layout block in `UserProfile.css` — were removed rather than worked around. Figure 8.3 (next chapter) shows the corrected Create Quiz page.

Root cause, in one sentence: plain, page-scoped CSS files sharing generic class names (`.badge` , `.logo` , `.left-panel` ...) combined with a router that loads every page's stylesheet unconditionally at start-up, so the last stylesheet loaded — not the page actually being viewed — silently won any naming collision.

CHAPTER 8

Results and System Screenshots

The screenshots in this chapter were captured directly from the running application (frontend on Vite, backend on Django, against a live SQLite database) rather than mocked up, and reflect the corrected UI described in Chapter 7.

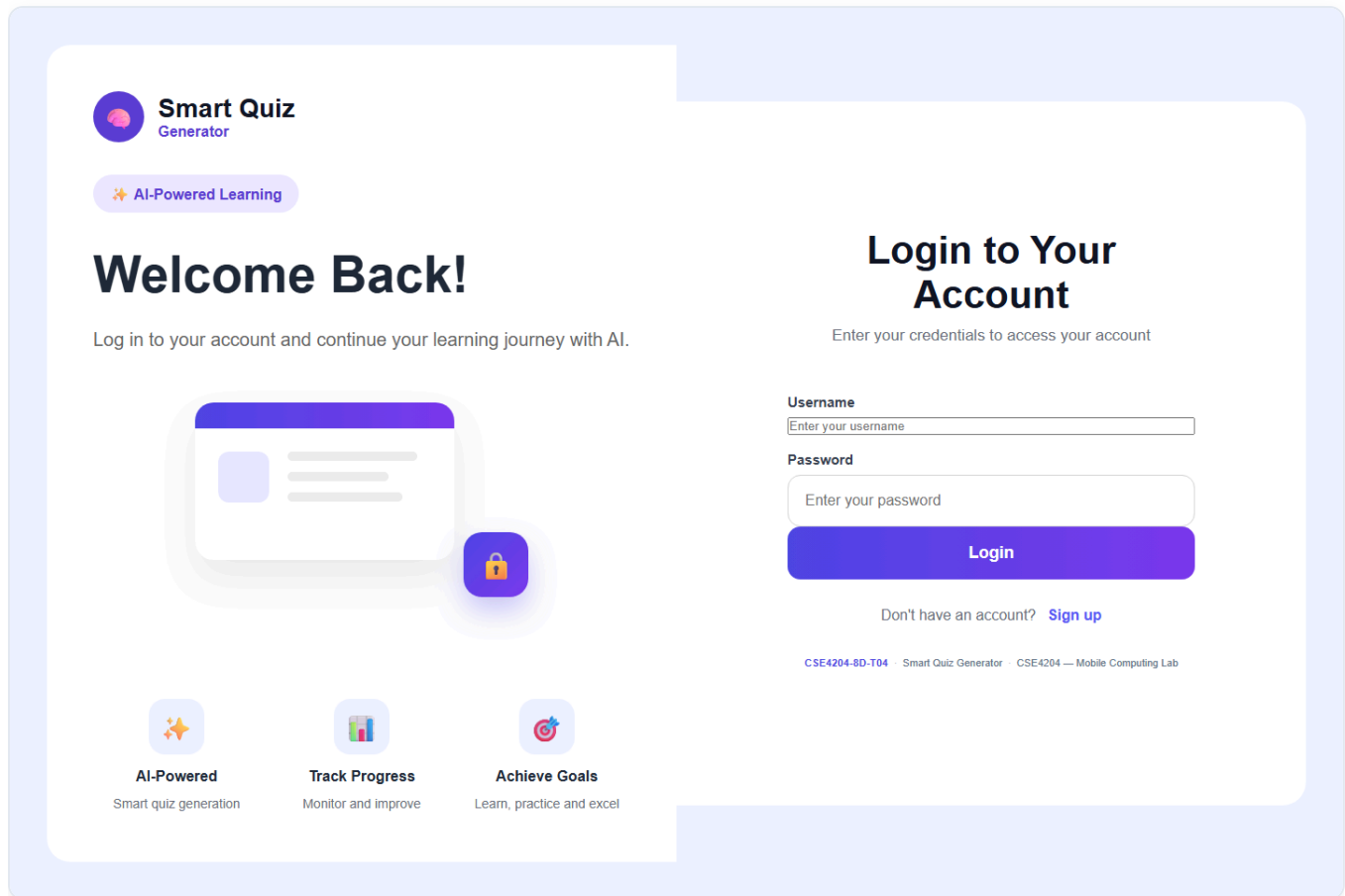
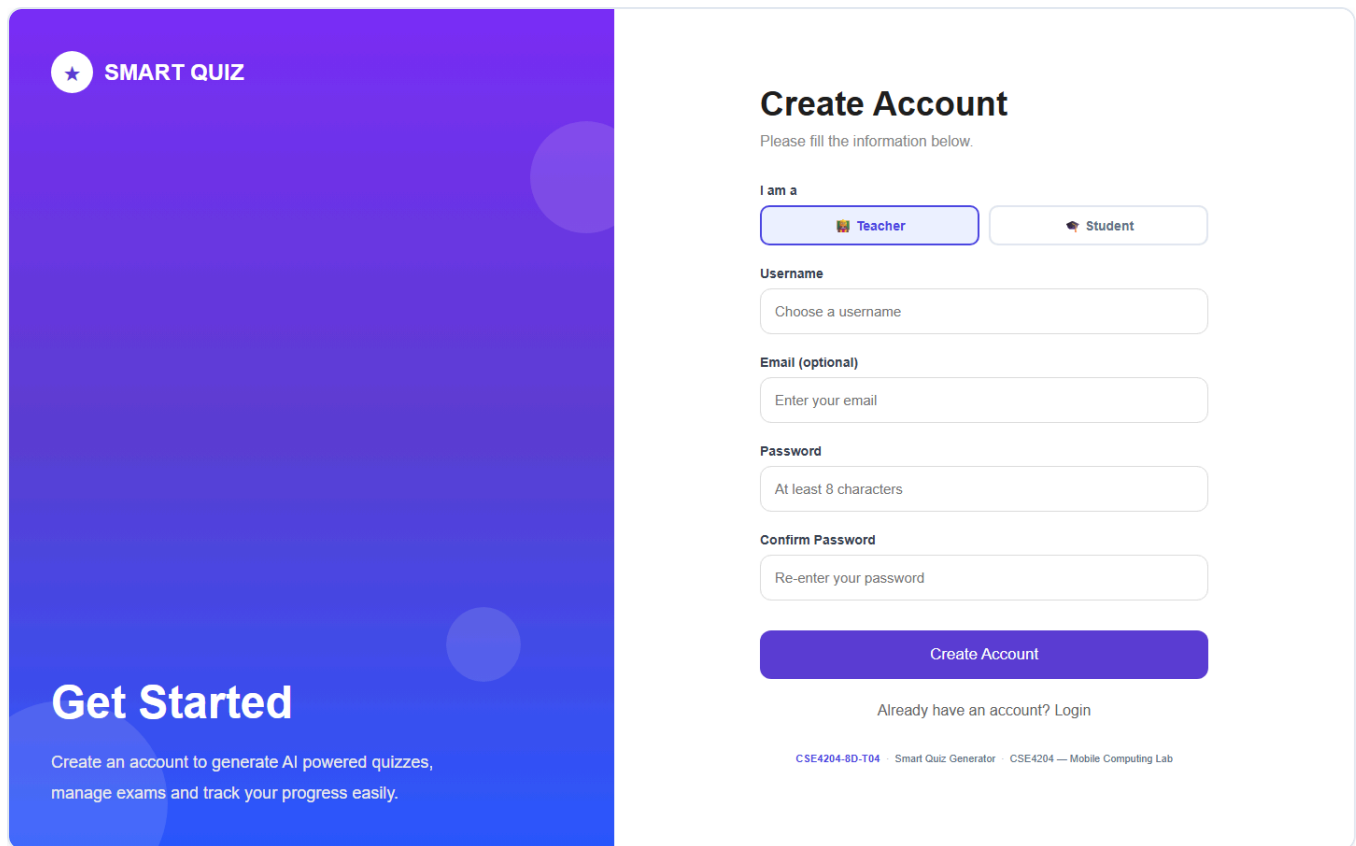


Figure 8.1. The login screen — split hero layout with feature highlights on the left and the credential form on the right.



The image shows a registration page for 'SMART QUIZ'. On the left is a purple sidebar with the 'SMART QUIZ' logo and a 'Get Started' section. The main area is white and contains a 'Create Account' form. The form includes a role toggle for 'Teacher' (selected) and 'Student', followed by input fields for 'Username', 'Email (optional)', 'Password' (with a hint 'At least 8 characters'), and 'Confirm Password' (with a hint 'Re-enter your password'). A 'Create Account' button is at the bottom of the form, with a 'Login' link below it. A footer at the bottom of the page contains the text 'CSE4204-8D-T04 · Smart Quiz Generator · CSE4204 — Mobile Computing Lab'.

 SMART QUIZ

Get Started

Create an account to generate AI powered quizzes, manage exams and track your progress easily.

Create Account

Please fill the information below.

I am a

 Teacher

 Student

Username

Email (optional)

Password

Confirm Password

Create Account

[Already have an account? Login](#)

CSE4204-8D-T04 · Smart Quiz Generator · CSE4204 — Mobile Computing Lab

Figure 8.2. Registration, with an explicit Teacher/Student role toggle collected at sign-up.

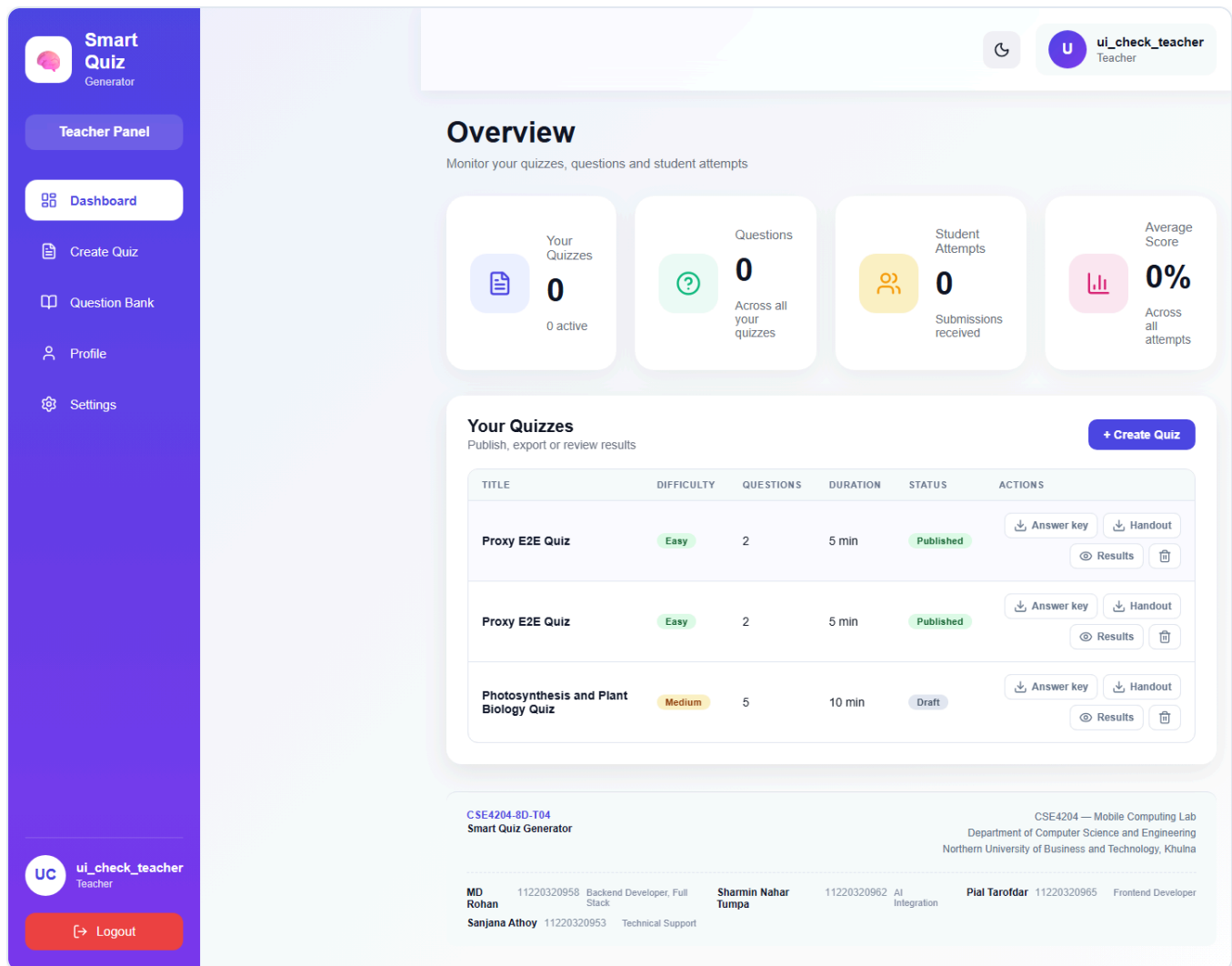


Figure 8.3. The teacher dashboard — quiz/question/attempt/average-score summary cards above a table of the teacher's own quizzes with per-row export and results actions.

 **Smart Quiz**
Generator

Teacher Panel

Dashboard

Create Quiz

Question Bank

Profile

Settings

UC

ui_check_teacher
Teacher

Logout

🌙

U **ui_check_teacher**
Teacher

Create New Quiz

Generate engaging quizzes in seconds with AI

1 Quiz Configuration

Topic *

e.g. Machine Learning basics

Source Document (Optional)

Upload a PDF, TXT, MD, CSV or JSON file and the questions will be generated from its contents.

Choose a file

Quiz Title (Optional)

Leave blank to let the AI name it

Syllabus / Context (Optional)

Paste the material the questions should be drawn from.

Difficulty Level

Easy Medium Hard

Number of Questions

Duration (minutes)

5 10

AI Provider

Server default

Additional Instructions (Optional)

e.g. Focus on practical applications, include real-world examples...

Generate Quiz

AI may generate inaccurate information. Please review before use.

2 Preview

Nothing generated yet

Fill in the configuration and hit "Generate Quiz". Your questions will appear here for review.

CSE4204-8D-T04
Smart Quiz Generator

MD Rohan

11220320958Backend Developer, Full Stack

Sanjana Athoy

11220320953Technical Support

Sharmin Nahar Tumpa

11220320962AI Integration

Pial Tarofdar

11220320965Frontend Developer

CSE4204 — Mobile Computing Lab
Department of Computer Science and Engineering
Northern University of Business and Technology, Khulna

Figure 8.4. The Create Quiz page after the layout fix described in Section 7.3 — the configuration form and the live preview pane now render correctly side by side.

Northern University of Business and Technology, Khulna

Page 28 of 39

Smart Quiz

Teacher Panel

- Dashboard
- Create Quiz
- Question Bank**
- Profile
- Settings

uL_check_teacher Teacher

Question Bank

Manage all quiz questions from one place.

Total Questions
9

Quizzes
3

With Explanation
9

Avg / Quiz
3

Search question ...

All quizzes

+ Add Question

#	Question	Quiz	Options	Answer	Action
1	Question 17 <i>Because</i>	Proxy E2E Quiz	A. Alpha B. Beta C. Gamma D. Delta	B	
1	Question 17 <i>Because</i>	Proxy E2E Quiz	A. Alpha B. Beta C. Gamma D. Delta	B	
1	What are the primary energy-carrying products of the light-dependent reactions of photosynthesis that are then used in the Calvin cycle? <i>The light-dependent reactions convert light energy into chemical energy in the form of ATP and NADPH, which are crucial for powering the synthesis of sugars in the Calvin cycle.</i>	Photosynthesis and Plant Biology Quiz	A. Glucose and Oxygen B. ATP and NADPH C. Carbon Dioxide and Water D. Starch and Water	B	
2	Question 27 <i>Because</i>	Proxy E2E Quiz	A. Alpha B. Beta C. Gamma D. Delta	A	
2	Question 27 <i>Because</i>	Proxy E2E Quiz	A. Alpha B. Beta C. Gamma D. Delta	A	
2	Which enzyme is responsible for the initial fixation of carbon dioxide into an organic molecule during the Calvin cycle? <i>Rubulose-1,5-bisphosphate carboxylase/oxygenase (Rubisco) catalyzes the reaction where CO₂ combines with RuBP, initiating the Calvin cycle.</i>	Photosynthesis and Plant Biology Quiz	A. Amylase B. ATP synthase C. Rubisco D. Pepsin	C	
3	Stomata are small pores on the surface of plant leaves. Their primary functions include: <i>Stomata regulate the exchange of gases, allowing CO₂ to enter for photosynthesis and O₂ to exit, while also controlling the release of water vapor through transpiration.</i>	Photosynthesis and Plant Biology Quiz	A. Water absorption from the atmosphere and nutrient transport B. Light absorption for photosynthesis and ATP synthesis C. Gas exchange (CO ₂ intake, O ₂ release) and transpiration D. Glucose production and storage	C	
4	The oxygen gas released as a byproduct during photosynthesis primarily originates from the splitting of which molecule? <i>During the light-dependent reactions, water molecules are split (photolysis) to provide electrons and protons, releasing oxygen as a byproduct.</i>	Photosynthesis and Plant Biology Quiz	A. Carbon dioxide B. Glucose C. Water D. Atmospheric oxygen	C	
5	An increase in which environmental factor would generally lead to an increased rate of photosynthesis, assuming other factors are not limiting? <i>Light provides the energy for the light-dependent reactions. Up to a certain saturation point, increasing light intensity directly increases the rate of these reactions and thus the overall rate of photosynthesis.</i>	Photosynthesis and Plant Biology Quiz	A. Atmospheric nitrogen concentration B. Soil pH C. Light intensity D. Humidity	C	

CSE4204-8D-T04
Smart Quiz Generator

CSE4204 — Mobile Computing Lab
Department of Computer Science and Engineering
Northern University of Business and Technology, Khulna

MD Rohan 11220320958 Backend Developer, Full Stack
Sanjana Athoy 11220320953 Technical Support

Sharmin Nahar Tumpa 11220320962 AI Integration

Pai Taruldar 11220320965 Frontend Developer

uc uL_check_teacher Teacher

+ Logout

Figure 8.5. The cross-quiz question bank, with aggregate stat tiles, search/filter controls, and per-question answer highlighting.

Smart Quiz
Generator

Teacher Panel

Dashboard

Create Quiz

Question Bank

Profile

Settings

uc

ui_check_teacher
Teacher

Logout

Settings

Manage your Smart Quiz Generator preferences and system settings.

Profile Settings

Update your personal information.

Full Name

Pial Tarofder

Email Address

pialtarofder00@gmail.com

Phone Number

+880 1234-567890

Department

Computer Science & Engineering

Update Profile

Appearance Settings

Customize the appearance of your dashboard.

Dark Mode

Enable or disable dark mode.

Theme Color

Select your preferred dashboard color.

Blue

Font Size

Choose your preferred text size.

Medium

Notification Settings

Choose which notifications you would like to receive.

Email Notifications

Receive important updates via email.

Quiz Notifications

Receive alerts when quizzes are created or updated.

System Notifications

Receive maintenance and system update alerts.

Weekly Reports

Receive weekly performance summaries.

System Settings

Configure default Smart Quiz Generator options.

Default Quiz Time

Set the default duration for every quiz.

30 Minutes

Default Passing Marks

Select the minimum passing percentage.

40%

Auto Save Quiz

Automatically save quizzes while editing.

Shuffle Questions

Randomize question order for every quiz.

Save Changes

Figure 8.6. The account settings page — profile, appearance, notification, and system preference sections.

Northern University of Business and Technology, Khulna

Page 30 of 39

Smart Quiz Generator

Student Portal

Quizzes

My Attempts

Profile

Settings

ui_check_student
Student

Available Quizzes
Pick a quiz and test what you know

Available
2
Published quizzes

Quizzes Taken
0
0 attempts

Average Score
0%
Across your attempts

Best Score
0%
Your personal best

Proxy E2E Quiz
2 questions
Start

Proxy E2E Quiz
2 questions
Start

CSE4204-8D-T04
Smart Quiz Generator

CSE4204 — Mobile Computing Lab
Department of Computer Science and Engineering
Northern University of Business and Technology, Khulna

MD Rohan 11220320958 Backend Developer, Full Stack
Sanjana Athoy 11220320953 Technical Support
Sharmin Nahar Tumpa 11220320962 AI Integration
Pinal Tarofdar 11220320965 Frontend Developer

Logout

Figure 8.7. The student portal's quiz list, showing only currently active (published) quizzes.

 **Smart Quiz**
Generator

Student Portal

Quizzes

My Attempts

Profile

Settings

UC

ui_check_student
Student

Logout

Proxy E2E Quiz

2 questions · Easy · answered 0/2

05:00

1 Question 1?

A Alpha

B Beta

C Gamma

D Delta

2 Question 2?

A Alpha

B Beta

C Gamma

D Delta

2 question(s) still unanswered.

Submit Quiz

CSE4204-8D-T04
Smart Quiz Generator

CSE4204 — Mobile Computing Lab
Department of Computer Science and Engineering
Northern University of Business and Technology, Khulna

MD Rohan

11220320958 Backend Developer,
Full Stack

Sanjana Athoy

11220320953 Technical
Support

Sharmin Nahar Tumpa

11220320962 AI
Integration

Pial Tarofdar

11220320965 Frontend
Developer

Figure 8.8. The timed quiz-taking interface — a live countdown, per-question progress, and a running "answered X/Y" count.

Smart Quiz Generator

Student Portal

- Quizzes
- My Attempts
- Profile
- Settings

50%
Well done!
You scored 1 out of 2 on "Proxy E2E Quiz".

[My attempts](#) [Back to quizzes](#)

Review
See which answers were right and why

1 Question 1? ✗
You chose A: Alpha Correct: B: Beta
Why: Because.

2 Question 2? ✓
You chose A: Alpha
Why: Because.

CSE4204-8D-T04 Smart Quiz Generator

CSE4204 — Mobile Computing Lab
Department of Computer Science and Engineering
Northern University of Business and Technology, Khulna

MD Rohan	11220320958 Backend Developer, Full Stack	Sharmin Nahar Tumpa	11220320962 AI Integration	Pinal Tarofdar	11220320965 Frontend Developer
Sanjana Athoy	11220320953 Technical Support				

[Logout](#)

Figure 8.9. The result screen shown immediately after submission — automatic scoring, plus a full review of which answers were right or wrong, with explanations.

The screenshot displays the 'Smart Quiz Generator' Student Portal. The left sidebar contains navigation links: 'Student Portal', 'Quizzes', 'My Attempts' (highlighted), 'Profile', and 'Settings'. The user is logged in as 'ui_check_student'. The main content area is titled 'My Attempts' with the subtitle 'Every quiz you have submitted'. It shows a single attempt for the 'Proxy E2E Quiz' on 14/08/2026, with a score of 50% (1/2 correct). Below this, the quiz details for 'CSE4204-8D-T04 Smart Quiz Generator' are listed, including the course name, department, and a list of staff members: MD Rohan, Sanjana Athoy, Sharmin Nahar Tumpa, and Pinal Tarofdar.

Smart Quiz Generator
Student Portal

My Attempts
Every quiz you have submitted

1 attempt
Personal best: 50%

Take another quiz

Proxy E2E Quiz
14/08/2026, 20:44:37

50%
1 / 2 correct

CSE4204-8D-T04
Smart Quiz Generator

CSE4204 — Mobile Computing Lab
Department of Computer Science and Engineering
Northern University of Business and Technology, Khulna

MD Rohan	11220320958	Backend Developer, Full Stack	Sharmin Nahar Tumpa	11220320962	AI Integration	Pinal Tarofdar	11220320965	Frontend Developer
Sanjana Athoy	11220320953	Technical Support						

UC **ui_check_student**
Student

[Logout](#)

Figure 8.10. The student's personal attempt history, populated with the attempt captured in Figure 8.9.

CHAPTER 9

Challenges and Solutions

Keeping two AI providers behind one contract

Gemini and Claude return question data in different shapes and with different failure modes. The solution was to push all provider-specific parsing into two isolated modules (`gemini.py` , `claude.py`) and validate both against one shared schema before anything reaches the database, so the rest of the application — persistence, PDF export, the frontend preview — never has to know which provider was used.

Never trusting the client with scoring or roles

Because the frontend already knows the correct answers once a quiz result is displayed, it would be easy for a naive implementation to let the client compute and submit its own score, or infer role from local state alone. Every scoring and permission decision was deliberately pushed server-side and re-derived from the database and the authenticated user's group membership on every request, even though this duplicates logic the frontend could technically also perform.

A CSS collision that only appeared once every page's styles were loaded together

As documented in Section 7.3, the most instructive bug encountered during this project was not in the backend or the AI pipeline at all, but a frontend styling collision that only manifested because the application loads every page's CSS unconditionally at start-up. It was a useful reminder that a component or page working correctly in isolation is not the same as it working correctly once composed into the full application, and that "it looks fine on the page I'm editing" is an insufficient test for a change to a globally-scoped stylesheet.

Making PDF exports match the live UI exactly

An early risk was that the PDF a teacher downloads could visually drift away from an offline preview tool used during AI-quality review, since the two would naturally be maintained separately over time. The fix was structural rather than procedural: both call the same `render_quiz_pdf()` function from `quiz_api/pdf.py` , so there is only one implementation to keep correct, not two to keep in sync.

CHAPTER 10

Conclusion and Future Work

10.1 Conclusion

Smart Quiz Generator delivers a complete, working answer to the problem stated in Chapter 1: it removes the manual burden of writing quiz questions by hand while keeping a human teacher firmly in control of what ultimately reaches students, it enforces a real, server-verified boundary between what a teacher and a student can each see and do, and it gives students immediate, explained feedback instead of a delayed grade. Every major subsystem described in this report — JWT authentication, role-based permissions, the quiz/ question/attempt data model, the dual-provider AI pipeline, the shared PDF renderer, and the React frontend — is implemented and was exercised end to end, both by an automated 28-step API test suite and by hands-on verification of the running application in a real browser, including the discovery and repair of a genuine layout regression on the project's own Create Quiz page.

10.2 Future Work

- **Production infrastructure.** Fronting the API with a reverse proxy/load balancer and adding a caching layer for read-heavy endpoints (quiz listings, stats) would matter once the system moves beyond development traffic.
- **Analytics dashboard.** The `/api/stats/` endpoint already aggregates per-user counters; a dedicated analytics view — score distributions, per-question difficulty inferred from real attempt data, cohort comparisons — is a natural extension.
- **Timed auto-submit hardening.** The client-side countdown could be backed by a server-enforced deadline so a manipulated client clock cannot extend a student's effective time limit.
- **Bulk question import/export** in a plain spreadsheet format, for teachers migrating an existing question bank into the system.
- **A system-administrator role**, anticipated in the original requirements but out of scope for this iteration, for institution-level user and quiz oversight.
- **Automated frontend regression testing.** The layout bug fixed in Chapter 7 was caught by manual, browser-driven verification; a small suite of automated visual/DOM assertions per page would catch the same class of cross-page CSS collision automatically on every future change.

REFERENCES

References

- [1] Django Software Foundation, *Django Documentation*, <https://docs.djangoproject.com/>
- [2] Encode OSS Ltd., *Django REST Framework Documentation*, <https://www.django-rest-framework.org/>
- [3] Jazzband, *django-rest-framework-simplejwt Documentation*, <https://django-rest-framework-simplejwt.readthedocs.io/>
- [4] Meta Platforms, Inc., *React Documentation*, <https://react.dev/>
- [5] Remix Software, Inc., *React Router Documentation*, <https://reactrouter.com/>
- [6] VoidQ / vitejs, *Vite Documentation*, <https://vitejs.dev/>
- [7] Google, *Gemini API Documentation*, <https://ai.google.dev/>
- [8] Anthropic, *Claude API Documentation*, <https://docs.anthropic.com/>
- [9] py-pdf community, *pypdf Documentation*, <https://pypdf.readthedocs.io/>
- [10] fpdf2 contributors, *fpdf2 Documentation*, <https://py-pdf.github.io/fpdf2/>
- [11] Team CSE4204-8D-T04, *Software Requirements Specification — Smart Quiz Generator*, project repository, 2026.
- [12] Team CSE4204-8D-T04, *Backend Progress Report — Smart Quiz Generator*, project repository, 2026.
- [13] Team CSE4204-8D-T04, *Entity-Relationship, Use Case, and Architecture Diagrams*, project repository, [diagrams/](#), 2026.

APPENDIX A

Team Contributions

Name	Student ID	Primary Contribution
MD Rohan	11220320958	Backend architecture and full-stack integration — Django/DRF project setup, JWT authentication, the quiz/question/attempt data model and permission system, the shared PDF export engine, and end-to-end wiring between the API and the React frontend.
Sharmin Nahar Tumpa	11220320962	AI integration — the Gemini and Claude provider clients, prompt design, response validation against the question schema, and the document-parsing pipeline that feeds AI generation.
Pial Tarofdar	11220320965	Frontend development — the React page and component library (Dashboard, Create Quiz, Question Bank, Student portal, Settings), shared layout/theme system, and role-based routing.
Sanjana Athoy	11220320953	Technical support — testing coordination, documentation, and diagram/report preparation across the project.

All four members additionally participated in cross-cutting quality assurance: exercising the API test suite, manually verifying frontend pages in-browser across both roles and themes, and reviewing one another's changes before they were merged.

APPENDIX B

Running the Project Locally

Backend

```
cd backend
python -m venv .venv && .venv\Scripts\activate
pip install -r requirements.txt
python manage.py migrate
python manage.py runserver
```

By default the backend uses a zero-setup SQLite database file. Setting `DB_ENGINE=mysql` (plus the corresponding `DB_*` environment variables) switches to the MySQL/MariaDB schema described in Chapter 4 without any code changes. AI generation requires at least one of `GEMINI_API_KEY` or `ANTHROPIC_API_KEY` to be set in `backend/.env`.

Frontend

```
cd frontend
npm install
npm run dev
```

The Vite dev server runs on port 5173 and proxies every `/api/*` request to the Django server on port 8000, so no CORS configuration is needed for local development.

Verifying the API independently

```
cd backend
python system_check.py
```

Runs the full diagnostic suite described in Section 7.2 against a locally running server and prints a PASS/FAIL/SKIP verdict for every step.